



AMX AI Voice Assistant

Intelligent real-time voice assistant

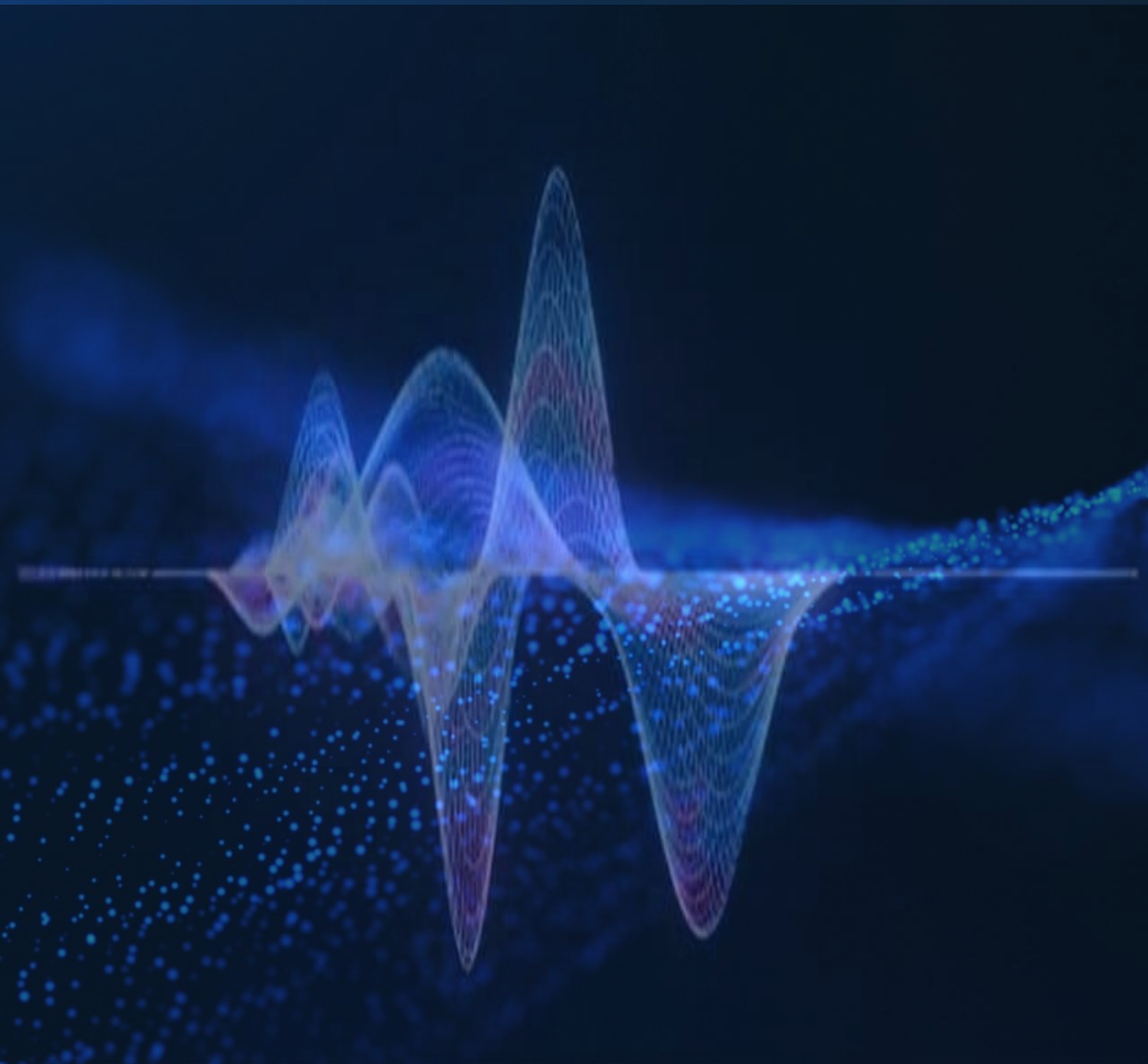
Creating the Future of Digital Engagement

USE CASE 4

AI-POWERED CUSTOMER SUPPORT ASSISTANT
FOR SMEs

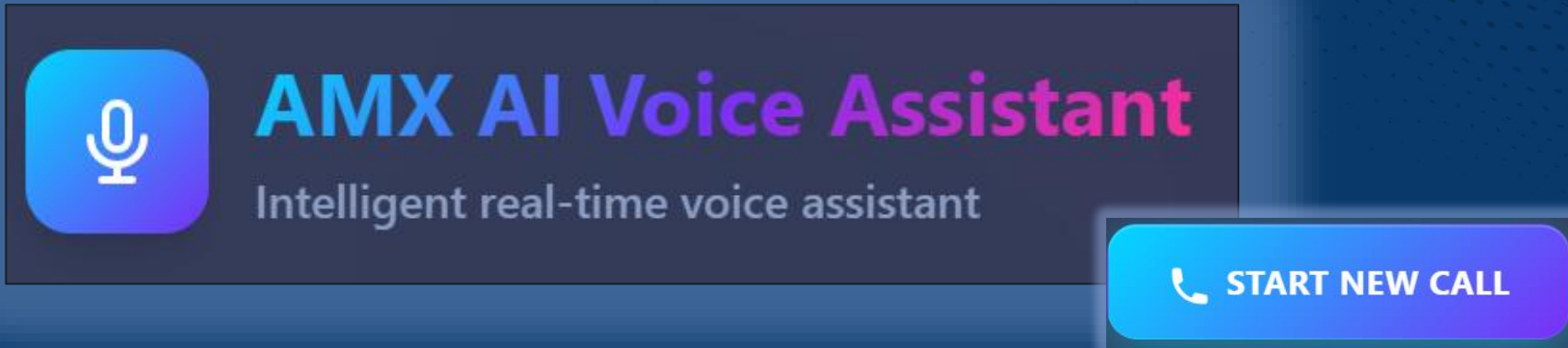
AZRA HIROLI
HASSAN RAZA
MANAL AKHTAR NOOR

AGENDA



- 1 Solution overview, USPs & POC
- 2 Technical Stack
- 3 Data Flow, Workflow & Methodology
- 4 System & Component Architecture
- 5 Development Options & Roadmap
- 6 Conclusion

The Solution



Conversational interface

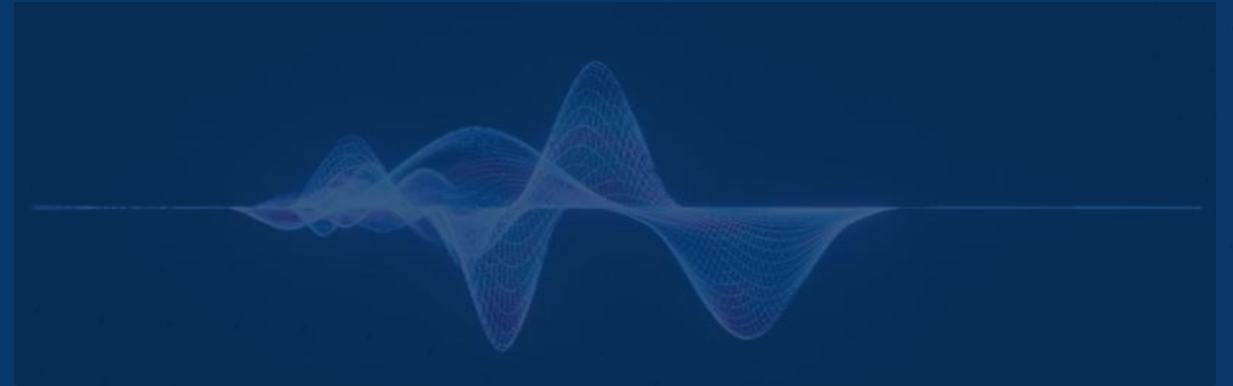
Multilingual Intelligence

Order Status
Check

Escalation to
Human

Callback
Booking

Support Ticket
Creation



Built with Next.js 14, TypeScript, and Vapi SDK

Key Design, USPs & POC

Client-Side Voice Processing

Database-First Approach

Security Considerations

Event-Driven Architecture

Progressive Enhancement

Stateless API Routes



Digital Transformation

Next-generation brand representation



Interactive Engagement

Real-time client conversations



Multilingual Support

Diverse market accessibility



Futuristic Appeal

Match brand's innovative stature

[AMX AI Voice Assistant](#)

TECHNICAL STACK

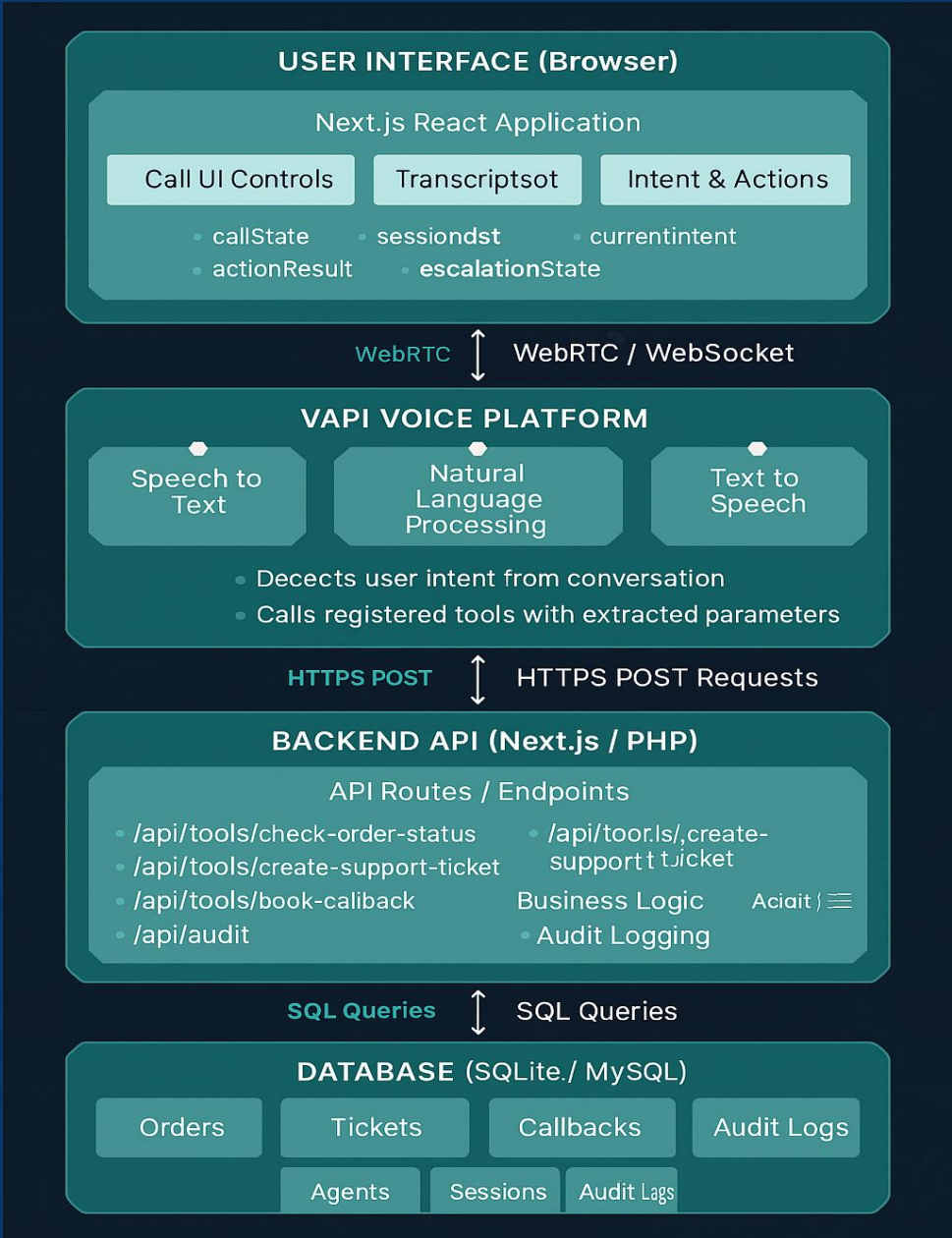
Frontend	Backend (Full-Stack Version)	Backend (PHP Version)	Development Tools
•Framework: Next.js 14 (App Router) •Language: TypeScript •Styling: Tailwind CSS + Custom CSS •State Management: React Hooks (useState, useEffect, useRef) •Voice SDK: Vapi Web SDK 2.3.0 •Icons: SVG inline components	•Runtime: Node.js 18+ •API Framework: Next.js API Routes •ORM: Prisma •Database: SQLite (dev) / MySQL (production) •Authentication: JWT (jose) + bcrypt	•Language: PHP 7.4+ •Database: MySQL with PDO •Authentication: JWT (Firebase PHP-JWT) •Password Hashing: password_hash() / password_verify()	•Package Manager: npm •Version Control: Git •Linting: ESLint •Formatting: Prettier (optional)

Architecture Overview



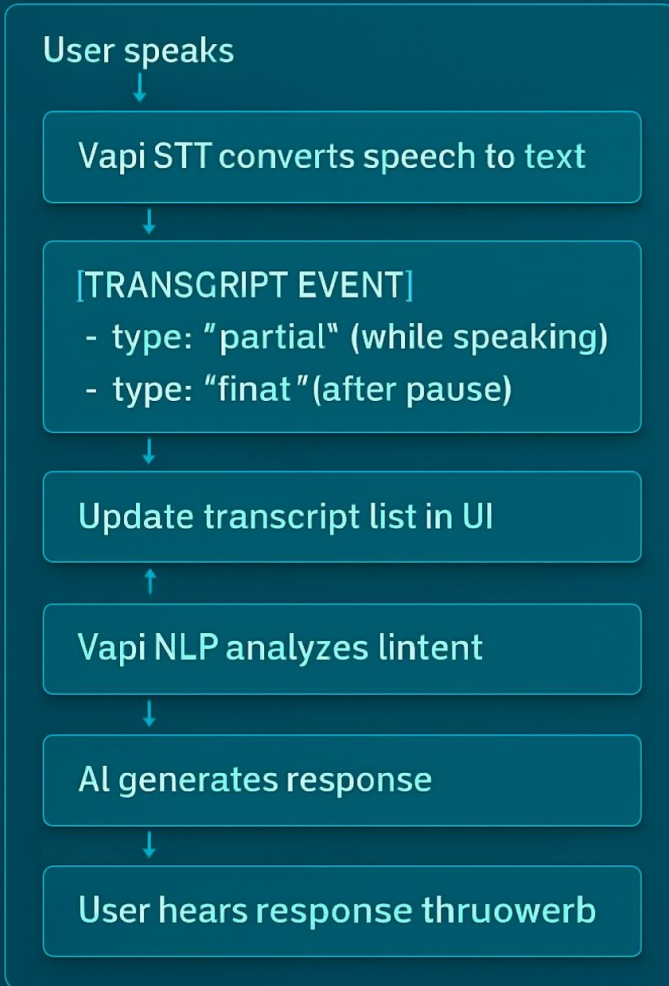
System Architecture

Component Architecture

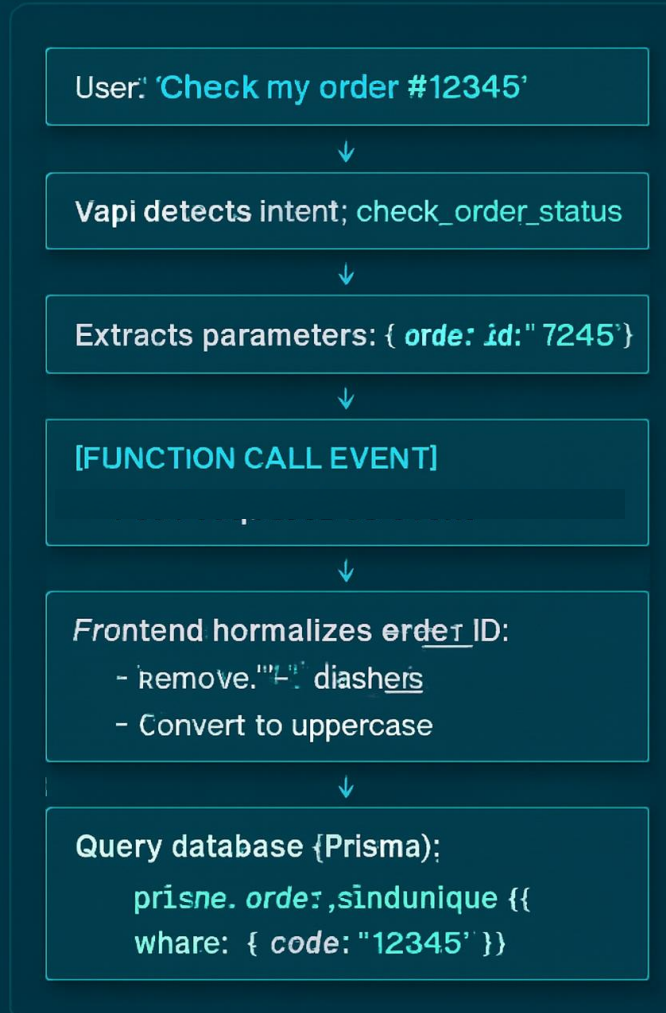


DATA FLOW & WORKFLOW

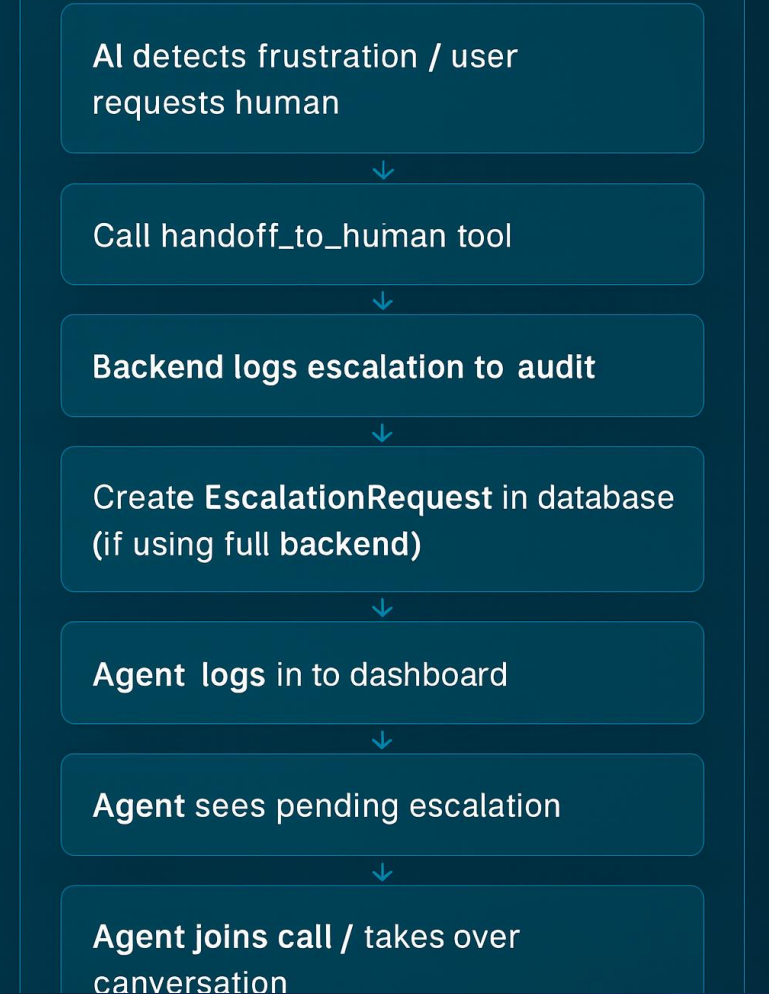
Voice Conversation Flow



Tool Invocation Flow



Escalation Flow



DATA FLOW & WORKFLOW

AMX AI AUDIT LOGGING FLOW

Any significant event occurs:

- Call started
- Intent detected
- Tool called
- Escalation requested
- Call ended

Call `logAudit()` function

Insert record into Audit table:

```
{ sessionId: "abc-123",  
  event: "tool_call",  
  payload: JSON.stringify(data),  
} timestamp: new Date()
```

Admin can view audit logs
in dashboard

METHODOLOGY & STEPS

Vapi Integration

1. Create Vapi assistant in Vapi dashboard
2. Configure system prompt with instructions and tool definitions
3. Initialize Vapi Web SDK in lib/vapiClient.ts
4. Register event handlers for: call-start: Initialize session
5. speech-start / speech-end: Update UI
6. message: Display transcripts (partial/final)
7. function-call: Handle tool invocations
8. call-end: Clean up session

PHASE 1

Project Setup

1. **Initialize Next.js project** with TypeScript, Tailwind CSS
2. **Configure Prisma** with SQLite/MySQL database
3. **Create database schema** for Orders, Tickets, Callbacks, Agents, Audit
4. **Seed database** with sample data for testing
5. **Set up environment variables** for Vapi API keys

PHASE 2

Backend API Development

1. Create API routes for each Vapi tool:
2. check-order-status: Query Orders table
3. create-support-ticket: Insert into Tickets table
4. book-callback: Insert into Callbacks table
5. handoff-to-human: Create escalation record
6. Implement audit logging utility
7. Add admin statistics endpoint for dashboard
8. Implement agent authentication (JWT + bcrypt)
9. Create escalation management endpoints

PHASE 3

METHODOLOGY & STEPS

Frontend UI Development

1. Design main call interface (app/page.tsx):
2. Call controls (Start/End/Escalate buttons)
3. Status indicators, Microphone visualization
4. Build transcript component with RTL support
5. Create intent card to show AI's understanding
6. Build action result display for tool outputs
7. Implement admin dashboard with stats and logs
8. Create agent login and dashboard interfaces

PHASE 4

PHASE 5

Styling & UX

1. **Apply futuristic design** with glassmorphism effects
2. **Add animations** & effects
3. **Implement responsive layout** for mobile/desktop
4. **Add loading states** and transitions
5. **Optimize for sub-500ms perceived latency**

Testing & Optimization

1. **Test order lookup** with various formats (#12345, AQ12345)
2. **Test support ticket creation** with different subjects
3. **Test callback booking** with date/time parsing
4. **Test escalation flow** end-to-end
5. **Test multilingual support** (English/Arabic)
6. **Optimize function calling** with clear prompts
7. **Test barge-in** (interrupting AI Deployment guides)

PHASE 6

PHASE 7

Deployment

1. **Static Export** (for shared hosting):
 1. Remove API routes
 2. Export as static HTML/CSS/JS
 3. Deploy to shared hosting
2. **Full-Stack Deployment** (for VPS/cloud):
 1. Deploy Next.js with Node.js
 2. Configure MySQL database
 3. Set up environment variables
 4. Enable HTTPS

DEVELOPMENT ROADMAP

MVP (Minimum Viable Product) - Week 1

- Basic call interface with Start/End buttons
- Real-time transcript display
- Order status checking
- Support ticket creation
- Callback booking
- Basic UI with Tailwind CSS

WEEK-1

WEEK-2

Enhanced Features - Week 2

- Futuristic UI design with animations
- Intent detection display
- Action result visualization
- Admin dashboard with statistics
- Audit logging system
- Microphone level meter

Advanced Features - Week 3

- Agent authentication system
- Escalation management
- Agent dashboard for handling escalations
- Session management with JWT
- MySQL integration for production

WEEK-3

Deployment Ready - Week 5

- Static export for shared hosting
- PHP version for compatibility
- Database migration scripts
- Setup wizard for easy installation
- Comprehensive documentation
- Deployment guides

WEEK-4

WEEK-5

Optimization & Polish - Week 4

- Improved function calling prompts
- Flexible order ID parsing
- Better error handling
- Responsive design for all devices
- Multi-language support (EN/AR)
- Barge-in optimization

FUTURE ENHANCEMENT

- Human look-alike AI
- Video call support
- Screen sharing for agents
- Analytics dashboard with charts
- Call recording
- Customer profile integration (CRM)
- Automated follow-ups
- Voice biometrics for authentication

DEVELOPMENT ARCHITURE

Option 1: Static Export (Shared Hosting)



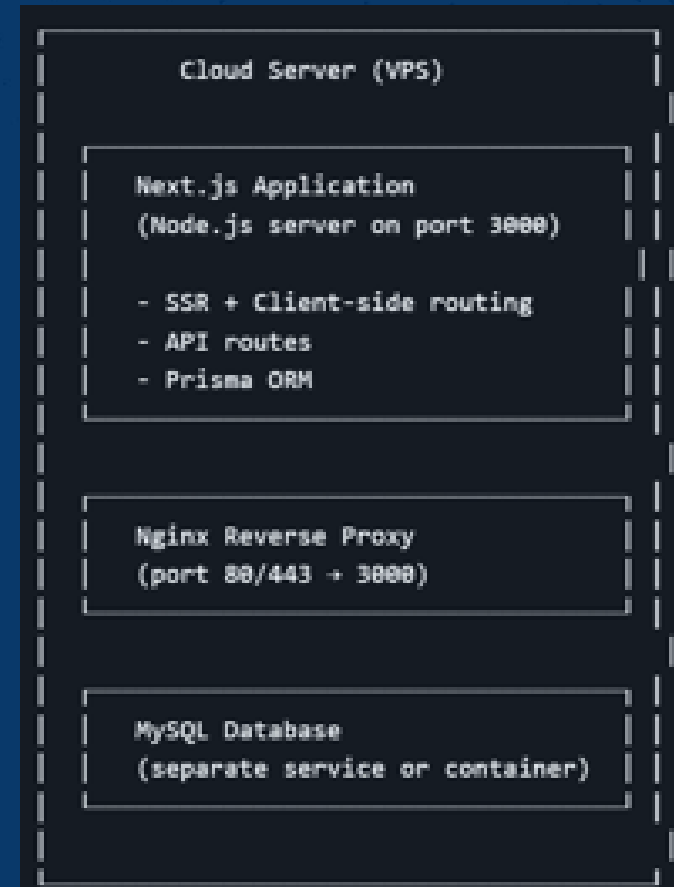
Advantages:

- Works on any shared hosting (no Node.js required)
- Simple deployment (FTP upload)
- Low cost
- Easy to maintain

Limitations:

- No server-side React rendering
- No advanced API features (middleware, etc.)
- Limited scalability

Option 2: Full-Stack Deployment (VPS/Cloud)



Advantages:

- Full Next.js features (SSR, ISR, etc.)
- Better performance
- Scalable architecture
- Advanced API capabilities

Requirements:

- Node.js 18+ installed
- Process manager (PM2)
- Nginx or Apache
- MySQL server

CONCLUSION

- Scalable, Modular, Production-Ready
- Flexible deployment
- Maintainable codebase
- Extensible features
- Intuitive user experience

“The combination of Vapi's voice platform with Next.js's modern React architecture creates a powerful, low-latency voice experience suitable for customer service, support automation, and conversational AI applications.”

Result

Pioneering the next generation of digital customer interaction

THANK YOU

[AMX AI Voice Assistant](#)